RAMAN ENTSIN

Chicago,IL • 815-296-6634 • entinraman86@gmail.com • https://www.linkedin.com/in/entinroman

SUMMARY

Full Stack / Frontend Software Engineer with 7 years of hands-on experience designing, developing, and scaling high-performance applications across fintech, HRM, and e-commerce domains. Skilled in both frontend (React, Next.js, Redux, Redux Toolkit, Redux-Saga, TypeScript, JavaScript, MobX, SCSS, Tailwind CSS, Material UI, Styled-Components, Framer Motion, Chart.js, Recharts) and backend (Node.js, ASP.NET Core Web API, Java, Spring Boot, Python/Django, Flask, FastAPI, Node.js, Express.js, Kafka, RabbitMQ, Amazon SQS, gRPC, GraphQL, REST APIs) development. Strong expertise in microservices architecture, event-driven systems, and authentication/authorization (OAuth 2.0, OIDC, RBAC).

Proficient with modern DevOps and cloud practices including Docker, CI/CD (GitHub Actions, Jenkins, Vite, Webpack, Husky, Eslint, Prettier), Ansible, Helm, and Azure Cloud services (App Service, Key Vault, Storage). Experienced with SQL (PostgreSQL, MySQL, MS SQL, Oracle) and NoSQL (MongoDB, Redis) databases, as well as observability and logging tools (Grafana, Serilog, NLog, Graylog, structured logging, monitoring pipelines).

Known for driving end-to-end delivery of complex applications, reducing defects, and improving performance and scalability by up to 40%. Adept at mentoring teams, leading cross-functional collaborations, implementing coding standards, and delivering secure, reliable software.

TECHNICAL SKILLS

- Languages & Frameworks: TypeScript, JavaScript, React, Next.js, React Native, Redux (Toolkit, Saga), Node.js, Express.js, ASP.NET Core Web API, Java (Spring Boot), Python (Django, Flask, FastAPI), MobX
- Frontend Tools: HTML5, CSS3, SCSS, Tailwind CSS, Material UI, Styled Components, Framer Motion, Chart.js, Recharts
- Backend & Messaging Systems: Apache Kafka, RabbitMQ, Amazon SQS, RESTful APIs, gRPC, GraphQL
- Databases: PostgreSQL, MySQL, Microsoft SQL Server, MongoDB, Oracle, Redis
- Testing & Quality: Jest, Vitest, React Testing Library, Xunit, Serilog, NLog, Graylog, Grafana
- DevOps & Infrastructure: Docker, GitHub Actions, Jenkins, Vite, Webpack, Husky, Eslint, Prettier, Ansible, Helm
- Practices & Patterns: Microservices, CQRS, Domain-Driven Design (DDD), SOLID principles

---

PROFESSIONAL EXPERIENCE

AGSR

Senior Full Stack Developer **Jan 2023 - Sep 2025**

- Frontend Experience

● Owned authentication UX end-to-end. I led the redesign and implementation of secure authentication interfaces (login, consent, multi-provider SSO) using OAuth 2.0 + OIDC. I standardized flows for token refresh, session timeout, and error recovery, documented edge cases (PKCE, invalid/expired tokens), and added clear user messaging. This reduced support tickets around login and produced a consistent SSO experience across projects.

● Performance wins that stuck. I audited bundles and dependencies, implemented route-level code splitting, tree-shaking, and dead-code elimination, and replaced heavy utilities with lighter alternatives. The result was a >40% decrease in initial bundle size and >60% reduction in code complexity, which translated to faster perceived load and simpler ongoing maintenance.

● Reusable UI at scale. I designed and published a shared component library (inputs, form builders, paginated tables, modals, skeletons) with versioning and changelogs. The library included accessibility-first patterns, theming tokens, and Storybook docs, cutting feature build time by ~2× and ensuring visual/behavioral consistency across teams.

● Real-time, conflict-free editing. I built an Edit Lock System that surfaces presence and lock state in real time via WebSocket channels. Users can see who's editing, request control, or work in a safe read-only mode. The UI coordinates with backend locking semantics to prevent overwrites, and gracefully handles reconnects and timeouts.

● Customer Relationship Portal. I delivered a portal for customers to submit feedback, report issues, and request features. It includes progressive form validation, attachment support, and status visibility. The front end integrates with backend workflows for triage and notification so customers are kept in the loop from submission to resolution.

● Quality bar and guardrails. I introduced a testing strategy using Jest and React Testing Library, focusing on critical flows (auth, save, submit, lock/unlock). I added linting, formatting, and commit hooks to prevent regressions, and established review checklists to keep accessibility and performance top of mind.

- Backend Experience

● Auth & identity services. I implemented Java (Spring Boot), Node.js/Express.js, and FastAPI services for authentication and authorization, handling OIDC discovery, token issuance/validation, refresh flows, and secure session lifecycles. I enforced scopes and claims at the API layer, standardized error contracts, and documented integration steps for other teams.

● Event-driven coordination with Kafka. I used Kafka topics to decouple user actions from downstream processing (e.g., edit-lock events, audit trails, notifications). Producers/consumers were idempotent and retry-aware, which improved resilience under bursty load and simplified recovery.

● Customer portal APIs. I built REST endpoints in Spring Boot and Django REST Framework for feedback/issue intake with validation, rate-limiting, and moderation hooks. Submissions fan out to notification services and analytics, while keeping PII handling compliant and auditable.
● Extended asynchronous services with FastAPI for real-time collaboration features and lightweight APIs.

● HRM platform contributions. As part of a distributed team, I contributed to service design, API contracts, and performance budgets. I collaborated closely with the front end to align pagination, filtering, and caching semantics, and added safeguards (idempotency keys, optimistic locking) around high-risk updates.

● Reliability and defect reduction. I resolved critical production defects across the stack by tightening validation, improving concurrency controls, and adding defensive checks. These changes improved stability and reduced user-reported issues by >40%.

- DevOps & Infrastructure

● CI/CD that developers trust. I set up CI/CD pipelines for UI and services with environment-aware builds, automated tests, and smoke checks. Rollouts included canary/feature flags for safer releases, and automated rollbacks on failed health checks.

● Standards and governance. I established coding standards, API guidelines (OpenAPI/Swagger), branch/merge conventions, and PR templates. Static analysis, test thresholds, and artifact provenance (build metadata) are now part of every merge.

● Observability where it matters. I instrumented hot paths (auth, save/submit, lock/unlock, portal intake) with structured logs and metrics, mapped them to dashboards/alerts, and added correlation IDs across services. This shortened MTTR and made incidents more diagnosable.

● Security hygiene. I enforced least-privilege access for services, rotated secrets, and added input/output validation at trust boundaries. Token scopes, CORS, and CSRF protections were audited and aligned with our API gateway rules.

- Collaboration, Leadership & Documentation

● Cross-team integration at pace. I coordinated with three teams to align API contracts, error codes, and release timing. We used shared mock servers during development to de-risk integrations and kept a transparent deployment calendar to avoid collisions.

● Mentorship & enablement. I mentored junior developers on testing, performance profiling, and review etiquette. I ran brown-bag sessions on OIDC flows, WebSocket patterns, and event-driven design to spread context and reduce silos.

● Clear, living documentation. I authored architecture overviews, sequence diagrams, component responsibilities, and runbooks. Docs are versioned with the code, include decision records (ADRs), and are referenced in onboarding checklists so new teammates can ship confidently in week one.

● Product partnership. I worked closely with the product to turn user pain points into measurable improvements (e.g., faster auth, clearer errors, conflict-safe editing). We aligned on SLAs, tracked UX KPIs, and used experiments/feature flags to validate changes before broad rollout.

WhiteSnake

Software Engineer **July 2018 - Dec 2022**

- Frontend Experience

● Commerce UI, tuned for mobile. Designed and implemented responsive, pixel-perfect interfaces for product listings, checkout flows, and customer dashboards using Next.js. Focused on touch targets, progressive disclosure, and skeleton states to keep interactions snappy on low-end devices, improving mobile UX by ~30% (task completion and abandonment rates).

● Speed + SEO via Next.js SSR/ISR. Implemented dynamic routing, SSR, and Incremental Static Regeneration (ISR) to serve fresh catalog data with CDN-cacheable pages. Added route-level code-splitting, image optimization, and prefetching for high-traffic paths, reducing initial load time by ~30% and increasing organic discoverability (crawlability and Core Web Vitals).

● Real-time tracking UI. Built an interactive item-tracking interface that renders tagged items on a map and updates order status in real time over WebSocket channels. Implemented connection resilience (backoff, re-subscribe) and UI diffing to keep updates smooth without over-rendering.

● Quality and safety nets. Established a front-end testing strategy with Jest and React Testing Library covering auth, cart, checkout, and order-status flows. Added linting, type-safety, and visual review checklists to prevent regressions in critical funnels.

- Backend Experience

● Order & catalog services. Developed backend services in Java (Spring Boot) and Node.js/Express.js for product management, inventory checks, cart orchestration, order processing, and secure payment lifecycles. Standardized REST contracts, implemented pagination/filtering semantics, and documented APIs for internal consumers.

● Payments done right. Completed integrations with VISA, PayPal, and MasterCard. Implemented secure payment APIs, idempotency keys for retry-safe operations, webhooks for asynchronous confirmations/refunds, and audit trails for charge disputes. Centralized error mapping to surface clear, actionable messages in the UI.

● Event-driven reliability with Kafka. Introduced Kafka to decouple checkout from downstream processors (fulfillment, notifications, analytics). Built retry-aware, idempotent consumers with dead-letter queues, improving order-status update reliability and smoothing traffic spikes during promotions and launches.

● Real-time pipelines. Backed WebSocket updates with lightweight geo-data services and order-event streams so customers see state changes as they happen (e.g., picked, packed, shipped), while preserving consistency between the UI and the source of truth.

- Testing, Observability & Delivery

● Full-stack test coverage. Extended the test strategy to backend unit/integration suites (auth, cart math, payment callbacks, webhooks), achieving 90%+ coverage on critical business flows. Added contract tests for payment gateways and synthetic journeys for checkout.

● Metrics that matter. Instrumented key endpoints and client events (cart add/remove, checkout start, payment authorize/capture, order status fan-out). Monitored p95 latency and error budgets for checkout and payment callbacks to catch regressions early.

● Faster, safer releases. Wired tests into CI, gated merges on coverage and static checks, and used staging smoke tests before production deploys. Feature-flagged risky changes (e.g., payment providers) to roll out gradually and reduce blast radius.

- Analytics & Growth

● Funnel visibility end-to-end. Implemented analytics tracking with Google Analytics, aligning frontend events with backend order states to create reliable conversion funnels (product view → add-to-cart → checkout → payment success). Built dashboards to diagnose drop-offs and attribute fixes to measurable gains.

● Data feedback loops. Streaming order events to analytics sinks alongside client-side signals to validate hypotheses about page performance, payment error rates, and the impact of UI changes on conversion.

- Collaboration & Documentation

● Clear contracts, fewer surprises. Authored API specs, sequence diagrams for payment flows (authorize, capture, refund), and webhook processing runbooks. Partnered with product, support, and finance to align edge-case handling (voids, partial captures, currency/rounding) and reduce ticket volume.

● Cross-team delivery. Coordinated with design on checkout UX (validation, error recovery, 3-D Secure handoffs) and with operations on fulfillment event semantics so status messages are meaningful to customers and actionable for internal teams.

EDUCATION

Master Degree in Computer Science, Minsk, Belarus
Belarusian State University of Informatics and Radioelectronics

Course Main Academy
The course includes:
1) Basic and advanced C#
2) Patterns (Factory method, MVC, MVVM)

Microservice Architecture microservices, Robot_dreams
The course includes:
1) Microservices patterns
2) Domain Driven Design
3) Microservices communication protocols
4) Resilience
5) Scaling